4.5 ppt

4.5 LLM Wiki：知识资产治理

一、 什么是 LLM Wiki？

上一节用 RAG 查找资料、根据证据回答问题；但有些问题需要反复整理多份资料之间的关系。

不保存整理结果，下次遇到类似问题，Agent 要重新建立这些关系。

定义：LLM Wiki 让模型根据原始资料生成、组织和维护知识页，把可以复用的总结、关联和分析保存下来，并随着来源变化更新。

知识资产治理要解决：怎样让知识持续积累，同时保证它有依据、能更新，并在允许的范围内使用。

二、 LLM Wiki 是一种工作模式

1. 与传统知识库相比，改变了什么

传统企业 Wiki 由人写页面、补链接、更新内容；原始资料不断增加，人未必有时间同步维护，页面容易落后于实际情况。

LLM Wiki 把这些动作交给 Agent：读取新资料、提取结论、找到相关页面、更新内容、补充引用和链接。

变化的是维护方式；来源、版本、审核和权限要求仍然需要保留——模型也可能总结错误，或者把计划写成已经完成的事实。

分工：工程师先规定整理和更新规则，Agent 执行，再通过检查决定生成内容能否使用。

这是一套工作流程，需要指令或调度程序触发。

2. Wiki、RAG、Memory各管什么

我们继续用登录项目区分。

机制	主要职责	登录项目里的例子
Working Memory / Checkpoint	保存当前任务的目标、进度和状态	已找到退出登录入口，尚未修改代码
Long-term Memory	保存以后可复用的偏好、约定和经验	过去修复中确认的到期边界经验
LLM Wiki	整理并维护有来源的项目知识	登录规则、实现位置、测试入口及其关系
RAG	为当前问题查找资料，供模型回答	找到登录知识页，必要时继续检索原文

内容可以重叠：测试入口既能保存在长期记忆里，也能写进 Wiki。区别在于怎样组织、更新和使用。

Wiki 负责整理和维护知识，RAG 负责为本次问题找到相关知识。两者可以配合：先检索知识页，再沿引用核对原文。

三、用登录项目搭出最小结构

职责明确以后，我们开始组织文件。先区分原始依据、生成内容和维护记录，再确定知识页写什么。

1. 原始资料、知识页和维护记录分开

下面是本节采用的目录结构。重点看用途和修改规则。

代码块

```
raw/          原始资料的版本快照
wiki/         模型生成并通过检查的知识页
staging/      尚未发布的生成结果
schema.md     维护规则：来源、版本、冲突、权限
manifests/    来源清单和知识页依赖关系
logs/         生成、校验、发布与失败记录
```

`raw/` 保留原始依据。资料更新时增加新快照，记录它替代哪个版本，旧版保留供追溯。

Agent生成或修改的页面先放进 `staging/` ，检查通过后再发布到 `wiki/` 。这样，未经核对的内容不会直接成为后续回答的依据。

`schema.md` 规定怎样整理：关键结论要给出处，资料不足怎样说明，冲突怎样处理，哪些内容需要批准。

`manifests/` 记录每页使用了哪些来源。登录规则变化后，程序根据这份记录找到需要重新核对的页面。

`logs/` 记录实际改了什么、检查是否通过、失败在哪里，方便排查维护过程中的问题。

代码块

```
请根据附件 corpus.zip，建立一个登录项目的 LLM Wiki 实验。

如果不能读取附件，请明确告诉我需要提供的本地路径，不要自行编造资料。

在当前目录创建 llm-wiki-demo；如果已经存在，先检查内容，不要覆盖已有文件。

完成以下准备工作：
1. 检查压缩包路径，拒绝绝对路径和越界路径。
2. 将 corpus 下的内容原样保存到 llm-wiki-demo/raw/v1/，保留相对目录。
3. 创建 wiki/、staging/、manifests/、logs/、reports/。
4. 创建 schema.md，规定：
   - raw 中的来源快照不由模型修改。
   - 新知识先生成到 staging，检查后才能进入 wiki。
   - 每条关键结论引用具体来源与位置。
   - 原文事实、源码观察、历史记录、需求草稿和推导分别表达。
   - 资料不足写待核对，不编造作者、版本、批准状态和执行结果。
   - 新来源进入后，检查直接和间接依赖的页面。
   - 合并重复内容时保留来源，未解决的冲突明确记录。
   - 权限由宿主程序执行，提示词和元数据不等于鉴权。
5. 创建 manifests/sources.json，登记每份来源：
   source_id、相对路径、实际 SHA-256、资料类型，以及原文提供的日期和状态。
   原文没有的信息记录为未知；区分资料版本与本次导入快照。
6. 创建空的 wiki/index.md 和 wiki/log.md。
7. 在 logs 中记录本次实际导入情况。

本步骤只准备资料和规则，不生成知识页，不运行或修改业务代码。
最后列出导入文件、无法解析的资料和创建的文件。
```

查看结果： `raw/v1/` 应包含Markdown、Python、CSV、HTML和PDF；来源清单中的哈希由程序计算。

2. 知识页写什么

知识页要回答三个问题：适用于哪里、结论是什么、依据在哪里。

页面头部记录页面和来源的身份、版本及权限，支持后续追溯。尖括号中的内容需要填入实际值。

代码块

```yaml
---
page_id: login-session
project: login_demo
page_version: 1
status: draft
owner: <知识负责人>
access_policy: <宿主程序确定的权限策略引用>
sources:
  - source_id: login-rules
    revision: <规则快照版本>
    content_hash: <规则原文实际哈希>
  - source_id: session-code
    revision: <源码快照版本或仓库提交>
    content_hash: <源码实际哈希>
  - source_id: test-guide
    revision: <测试说明版本>
    content_hash: <测试说明实际哈希>
---
```

来源版本说明页面依据哪份内容生成。

限字段关联程序执行的访问策略，字段本身不会阻止越权读取。

正文再把规则、实现和验证入口联系起来。

代码块

```
# 登录会话

## 适用范围
适用于 login_demo 的登录会话，不适用于支付会话。

## 当前规则
- 有效期为30分钟。[S1：有效期长度]
- 当前时刻等于到期时刻时，会话无效。[S1：到期边界]
- 用户请求触发续期，空闲会话不自动续期。[S1：有效期长度、续期]

## 实现与验证
- 时间判断位于 app/session.py，有效条件为 now < expires_at。
```

```
  [S2: is_session_valid]
- 令牌签发、校验和续期入口位于 app/auth.py。
  [S3: issue_token、check、touch]
- 测试入口为 python -m pytest tests/test_session.py -q。
  [S4: 测试入口]
- 当前资料未提供本次测试运行回执。

## 待核对事项
列出资料缺失、规则与实现差异，以及未解决的冲突。

## 来源
S1: <登录规则快照与小节>
S2: <session.py快照与函数>
S3: <auth.py快照与函数>
S4: <测试说明快照与小节>
```

这是目标模板，不是已经生成的页面。实际生成时，页头和依赖清单也要登记正文使用的全部来源。

生成后沿引用核对关键结论：实现位置看源码，测试入口看测试说明；没有本次执行回执，就不能把历史记录中的"测试通过"写成当前验证结果。

新人先通过知识页了解模块，再沿引用找到原文、源码和验证入口。

四、让AI生成，由程序控制发布

页面格式确定后可以让 Agent 生成知识，但生成完成不代表可以发布（例如 Agent 可能把退出登录需求写成已完成功能）。

流程分三步：**确定输入 → 检查草稿 → 提交发布**。

1. 确定输入，再生成草稿

宿主程序先确定项目和执行身份，检查读取权限，再固定本次来源版本，记录文件和内容哈希。

Agent 根据资料和维护规则，交付三类内容：

- 正文：整理出的规则、实现和验证入口。
- 来源映射：关键结论对应哪份资料、哪个位置。

- 待核对事项：资料缺失、冲突和未确认问题。

退出登录的例子：需求说明期望退出后令牌立即失效，源码却显示功能尚未实现。草稿应保留这个差异，说明还需要开发和验证。

代码块

```
请在 llm-wiki-demo 项目根目录操作。

先读取 schema.md 和 manifests/sources.json，再读取全部来源。
PDF使用可用的文本提取工具；表格保留列含义；HTML保留标题层级。
如果某份资料无法读取，记录缺口，不根据文件名推测正文。

生成以下草稿：
- staging/technical/login-session.md：登录规则、实现、续期和测试入口。
- staging/technical/token-lifecycle.md：令牌签发、签名校验、检查、续期、清理和退出。
- staging/comparisons/session-types.md：登录、支付、匿名浏览和记住我令牌的区别。
- staging/project/logout-plan.md：退出登录需求、当前实现、验收要求和未决问题。

生成时特别核对：
1. 登录会话30分钟，now == expires_at 时无效。
2. app/session.py负责时间判断，app/auth.py负责令牌生命周期。
3. logout()当前仍抛出NotImplementedError；v2.5.0 PRD是草稿。
4. 测试说明中的验收要求，不等于本次测试已经通过。
5. config.py虽然读取环境变量，但要根据实际调用关系判断它是否接入有效期计算。
6. 区分历史故障记录和当前源码；区分需求目标和已完成行为。
7. 不同会话类型不能混用有效期与续期规则。

每页包含：
页面身份、草稿状态、适用范围、关键结论、来源、待核对事项。

关键结论使用标准Markdown相对链接，标明来源小节或函数。
来源链接以将来发布到wiki后的目录位置计算。

另生成 staging/dependencies.json：
登记每页使用的来源ID、快照哈希、引用位置，以及依赖的其他知识页。
同时提供结论到证据的映射，证据摘录只保留核对所需的短句。

不修改raw，不发布到wiki，不运行业务测试。
最后列出草稿和发现的差异。
```

查看结果： raw/ 未修改，wiki/ 未发布。

2. 检查草稿，再决定是否发布

程序先检查来源、版本、引用和写入路径。

- 内容核对再检查：引用是否支持结论，有没有遗漏条件，有没有把建议、计划写成执行结果。
- 哪些页面可以自动发布、哪些需要负责人确认，由工程师预先规定；页面的阅读范围也要在发布时确定。

流程示意（不是完整可运行程序；重点：检查失败时保存什么，提交前还要确认什么）：

代码块

```
async def refresh_page(request, actor):
    snapshot = await sources.authorized_snapshot(request, actor)
    base = await pages.current_version(request.page_id)

    draft = await compiler.generate(snapshot, rules=request.rules)
    report = await validator.check(draft, snapshot, base)
    if not report.publishable:
        return await staging.save(draft, report)

    decision = await policy.decide(draft, report, actor)
    if not decision.allowed:
        return await staging.save(draft, decision)

    return await publisher.commit_if_current(
        draft=draft,
        source_snapshot=snapshot,
        expected_page_version=base,
        actor=actor,
    )
```

检查或发布策略未通过，就保存草稿和原因。

通过后，发布器再次核对来源、权限和页面版本，再提交更新。

3. 提交前，确认发布条件仍然有效

为什么要在提交前再检查：生成期间原文可能更新、权限可能撤销，其他任务也可能修改同一页。

知识页、来源依赖和导航索引要一起生效，避免页面更新了，其他记录仍停在旧版本。

模型负责整理内容，程序负责管理草稿、检查条件和控制发布。这就是 Harness 在知识维护中的分工。

代码块

```
请检查staging中的知识草稿，依据schema.md决定能否发布。

先做结构检查：
- 来源是否存在，实际哈希是否与登记值一致。
- 每条关键结论是否登记证据。
- 引用文件和引用位置是否有效。
- 页面ID是否重复，路径是否越界。
- 发布后的相对链接是否可以正确解析。

再逐条做内容核对：
- 引用是否支持结论。
- 是否遗漏否定词、条件或适用范围。
- 是否把需求草稿写成已经上线。
- 是否把历史测试记录写成本次测试结果。
- 是否把配置声明写成实际生效行为。

把结果写入reports/prepublish.md，区分：
结构检查结果、内容核对结果、未解决事项。

可以修复有原文依据的问题，但不能修改原文迎合结论。
缺资料或冲突未解决时，明确保留缺口；不能发布确定但无依据的事实。

检查通过的页面，按schema发布到wiki，并同步：
- manifests中的来源依赖和页面版本；
- wiki/index.md中的链接、摘要和页面状态；
- wiki/log.md中的操作记录。

提交前再次核对来源哈希和页面基线。
来源变化或同一页面被其他任务修改时，停止覆盖，报告原因。

最后说明：
哪些页面已发布，哪些仍为草稿，执行了哪些检查，哪些能力尚未实现。
不要把本地实验描述成已有生产级审批、权限或原子发布系统。
```

查看结果： `wiki/index.md` 能定位知识页，报告中有实际检查结果，而不只是“全部通过”一句话。

五、来源追踪：结论怎样回到原文

页面发布以后，用户看到“有效期30分钟”，应该能查到具体依据。

来源记录要包含资料、版本、位置和适用范围；作者、发布机构和时间，有就记录，没有就标为未知。

1. 关键结论对应具体依据

页面末尾的参考资料说明整页读过什么，结论旁的引用说明这句话由哪段原文支持。

2. 新页面仍要能追到原文

可以沿链接逐层追溯：对比页 → 知识页 → 原文。重要结论也可以直接引用原文。

3. 问答归档也要保留依据

有价值的问答可以存回Wiki。归档前，检查它是否有来源、是否适用于当前项目，以及是否与已有页面重复。

保存成文件，不会让模型输出自动成为事实。 归档要保留核对过的结论和依据。

代码块

```
请先读取schema.md、当前来源清单和wiki/index.md，再根据索引读取相关知识页。

回答以下新人问题：
1. 登录会话有效期多久，恰好到期是否有效？
2. 哪个模块判断时间，哪个模块管理令牌？
3. 用户请求怎样触发续期？
4. 退出登录是否已经实现？
5. 修改后应该用什么入口验证，目前有没有本次执行结果？

回答中的关键结论必须能追到原文。
涉及代码行为时核对源码，涉及需求状态时核对需求资料。
不要仅引用另一份AI总结作为最终依据。

将问答保存为wiki/queries/login-onboarding.md。
如果已有同类页面，更新它，不新建重复页面。

为该页登记原始来源和知识页依赖，更新索引并追加日志。
```

```
归档后，抽查三条关键结论，列出：
问答结论 → 知识页 → 原文位置。

不要执行业务修改或宣称测试已通过。
```

六、原文更新后，哪些知识必须跟着改

如果只保存新原文，Wiki 继续写 30 分钟，后续回答就可能使用旧规则。

更新必须从来源传递到知识页，再传递到检索结果。

1. 找到受影响页面

- 程序检测到来源内容或当前版本变化，保存新快照，再沿依赖找到登录知识页和登录、支付对比页。
- 这些页面先标为待复核；更新完成前，当前规则查询应读取新原文，并说明知识页尚未同步。

2. 按含义更新结论

- Agent 重读新规则、相关页面和源码，核对适用范围与生效时间，再修改对应结论。
- 新规则已生效但源码仍配置 1800 秒：页面要写清"规则要求 60 分钟，当前实现仍为 30 分钟，尚未同步"，不能为了页面一致就写成代码已经修改。
- 历史记录里的 30 分钟、支付会话的 5 分钟，不能一起替换。

3. 同步检索，保留历史

- 如果 RAG 检索这些页面，还要更新分块、检索索引和缓存，避免旧内容继续进入模型输入。
- 同一知识页保留稳定身份，用版本号区分内容；同步失败就标明状态并回退到当前原文。
- 旧版仍可用于历史查询，例如"上个月为什么 30 分钟后退出"，需要当时的规则和实现。
- 要区分版本、生效时间和入库时间：Git 保存修改历史，业务规则何时生效仍需单独记录。

代码块

```
现在进行一次版本更新。

请保留raw/v1全部内容，新增raw/v2/docs/session-rules.md。
这份新资料由我确认作为本实验的当前登录规则，内容如下：

```

```
# 登录会话规则 v2（更新）
适用服务：login_demo。
状态：本实验已批准。
生效条件：从本次教学激活开始适用，替代v1登录规则。

- 登录会话有效期为60分钟。
- now == expires_at时仍判定为无效。
- 空闲会话不自动续期，由用户请求触发刷新。
- 续期后的到期时间为请求到达时刻加60分钟。
- 不改变支付、匿名浏览和记住我令牌的规则。
- 本资料只改变规则，不证明源码已经修改或测试已经通过。

先登记新来源、实际哈希、替代关系和激活时间。
不要编造正式公司审批人或签字记录。

然后分两阶段操作：

阶段一：
根据依赖记录找到直接和间接受影响的知识页及归档问答。
标记为待复核，生成reports/update-impact.md。
记录更新前有哪些页面仍使用旧规则。

阶段二：
重新读取新规则及相关来源，更新受影响页面。
保留以下实际差异：
- 当前规则要求60分钟。
- 原始源码仍配置30分钟。
- FAQ、表格和其他原文若仍写30分钟，说明尚未同步，不修改它们。
- 退出登录实现状态没有因此改变。

检查通过后，更新页面版本、依赖、索引摘要和日志。
保留旧来源和页面历史，不能全局替换30或1800。

最后分别回答：
“当前批准的有效期是多少？”
“附件源码实际实现的有效期是多少？”

如果没有接入向量索引或缓存，明确记录未接入，不声称已经完成这些同步。
```

七、重复、过期和冲突，分别怎样处理

资料不断增加以后，除了版本更新，还会遇到重复、过期和冲突。它们的处理方式不同。

1. 重复：合并结论，保留来源

正式文档和需求记录都写了登录规则，可以把相同结论集中到一页，同时保留来源：正式文档说明规则，需求记录解释背景。

合并前核对项目、版本和适用范围。文字相似但对象不同，不能直接去重。

2. 过期：退出当前查询，保留历史用途

被替代的规则、旧接口和旧测试命令，要标明状态，从默认当前查询中排除。历史查询需要时，再按时间读取。

外部调研可以记录观察时间和复核期限。超过期限意味着需要核对，不代表一定错误。

3. 冲突：核对状态和权威来源

沿用前面的60分钟新规则。如果后来一份会议记录建议改为90分钟，这属于未批准建议，不能覆盖当前规则。

如果两份已批准规则在同一范围、同一生效期间给出不同值，就记录双方依据，交给负责人确认。确认前明确说明冲突。

判断不能只看文件新旧，也不能按数量投票。新导入的可能是旧资料，多篇转述也可能来自同一个错误来源。

模型帮助发现差异、整理证据，最终采用哪条结论，要按规定的权威规则和确认流程决定。

代码块

```
请对当前Wiki做一次维护巡检。

先新增一份教学资料raw/discussions/ttl-proposal.md：
标题：登录有效期讨论
内容：有人建议把登录会话有效期改为90分钟，但尚未批准，也未生效。
这份资料只能作为讨论记录，不能替代当前60分钟教学规则。

读取schema、来源清单、知识页和依赖记录，检查：

1. 重复：
是否存在表达相同知识的重复页面。
提出合并建议，同时保留来源、版本和适用范围。

2. 过期：
是否还有页面把30分钟写成当前批准规则。
区分旧规则、当前源码行为和历史记录，不能全部标成错误。
```

```
3. 冲突：
区分90分钟建议、60分钟当前规则、30分钟源码观察。
说明哪些是状态不同，哪些是规则与实现未同步。
真正无法解释的冲突，保留双方证据，不自行选定答案。

4. 关联：
是否有知识页未进入索引，引用失效，或者依赖记录遗漏。

先生成reports/lint.md，不直接删除或合并页面。
每项写清：页面、问题、证据、影响和建议动作。
没有发现问题时，写本次未发现，不宣布知识库永久正确。
```

查看结果：90分钟不会变成现行规则；源码30分钟被识别为实现差异，而不是直接删除。

八、权限在哪些环节执行

维护 Agent 可能同时读取公开说明和内部安全设计。如果把综合总结公开，即使没有复制原文，也可能泄露内部信息。

权限要贯穿读取、生成、发布和使用。

1. 读取时检查执行身份

宿主程序先确认维护 Agent 可以读取哪些资料，再交给模型。

检查范围包括正文、附件、图片、标题和元数据。

2. 发布时确定可见范围

派生知识不能因为经过总结，就扩大访问范围。

最小方案按整页授权：读者满足所有来源的访问条件，才能读取这页。

公开介绍单独使用公开来源生成；受限内容对外发布，按规定脱敏和审批。

权限由可信程序确定，模型不能自行改成公开。

3. 使用和撤权时继续检查

检索结果和文件工具读取都要检查权限，避免搜索受限，直接读取却能绕过。

索引、摘要、反向链接、导出和缓存也遵守同样的规则。

来源撤权后，根据依赖调整知识页权限，让相关索引和缓存失效。如果内容被保存为长期记忆，也要同步处理。

读取来源、生成内容、发布页面和阅读页面，是不同的操作，需要分别授权。

代码块

```
请检查这个Wiki实验的权限边界。

先确认实际已有的身份、文件工具、检索、发布和导出机制。
不要根据schema中的文字推断系统已经实现鉴权。

生成reports/access-review.md，逐项说明：
- 来源读取是否检查身份。
- 生成页面的可见范围由谁确定。
- 发布是否检查访问策略。
- 索引、摘要、文件读取和导出是否执行相同限制。
- 来源撤权后，依赖页和缓存怎样处理。

如果已有可执行权限控制：
使用授权身份和未授权身份分别读取同一受限来源、派生页和索引摘要；
记录实际允许或拒绝结果。

如果没有：
明确标为未实现。
给出最小接入位置和验收用例，不伪造拒绝日志，
不把“模型遵守提示词”算作权限隔离通过。

不要自行改变真实目录权限，也不要假装切换身份。
```

查看结果：报告清楚区分“已有能力”和“尚未实现”。没有鉴权时，结论应是待实现。

九、怎样检查知识质量

流程和权限确定以后，最后用验收结果判断知识是否可用。

1. 从四个维度检查

维度	检查什么	登录项目里的例子
完整性	关键内容是否齐全，缺口是否明确	有效期、边界、实现位置和测试入口
可信度	引用是否支持结论	回到原文核对事实、推导和建议
时效性	当前查询是否使用有效版本	新规则生效后，不再把旧规则当现行事实
可检索性	不同问法能否找到相关知识	用“登录状态老是掉”“会话过期”测试

2. 程序查结构，模型辅助查含义

程序检查来源、链接、版本、依赖和权限。模型辅助发现语义冲突、条件遗漏和缺少的关联。

需要补充信息时，先搜集可靠资料并保存来源，再更新页面。没有资料，就保留待核对事项。

3. 用比较结果判断收益

Wiki可能减少重复整理，也增加生成、更新、审核和同步的开销。

用同一组问题比较两种方案：直接检索原文回答，以及先检索Wiki、必要时回看原文。分别检查答案、引用、维护费用和回答耗时。

没有实际比较，就不能宣布Wiki一定更省钱或更准确。知识页正确，回答模型仍可能误读，所以回答侧检查也要保留。

代码块

```
请对本次Wiki操作做最终验收。

先读取当前来源清单、知识页、维护报告和实际操作日志。
按四个维度检查，并生成reports/acceptance.md：

1. 完整性：
登录有效期、到期边界、续期、实现位置、退出状态和测试入口是否有依据；
缺失项是否明确。

2. 可信度：
抽查至少六条关键结论，列出结论、原文位置和支持情况；
区分原文事实、推导、建议与执行结果。

3. 时效性：
分别提问“当前批准规则”“当前源码行为”“v1历史规则”；
检查答案是否混用版本和状态。

4. 可检索性：
使用“登录状态老是掉”“会话过期”“退出后令牌还能用吗”等问法；
```

```
通过实际可用的索引或搜索定位页面，记录找到的路径和依据。
没有检索工具时说明使用索引导航，不虚报召回率。

另外核对：
- 新规则是否更新到相关知识页和归档问答。
- 90分钟建议是否仍保持未批准状态。
- 是否存在未经证据支持的“已上线”“已修复”“测试通过”。
- 权限报告是否如实标记未实现能力。

每项输出：通过、未通过或未验证，并说明证据。
模型usage或费用不可获得时留空，不填零。
本次未运行原文RAG对照实验，不宣布Wiki更准确或更便宜。

最后给出本次实际完成的操作，以及下一步需要修复的具体问题。
```

小结

这一节把登录规则、实现和测试之间的关系，整理成可以复用和维护的知识页。能否放心使用，要检查四件事：

1. **有依据**：关键结论能追到原文和版本。
2. **能更新**：来源变化后，知识页、索引和缓存一起同步。
3. **不隐藏冲突**：区分事实与建议、当前与历史，保留未确认的差异。
4. **权限有效**：程序控制读取、发布和使用范围，并用实际回执验证。

保存下来只是起点。能够追溯、更新和受控使用，知识页才有条件成为后续 Agent 可复用的知识资产。